SMART STUDY PLANNER

PROJECT REPORT

VIT BHOPAL UNIVERSITY

School of Computing Science and Engineering

Bhopal-Indore Highway, Kothrikalan, Sehore

Madhya Pradesh - 466114

CSA1021 – INTRODUCTION TO PROBLEM SOLVING AND PROGRAMMING

REG. NO.: 26BCE10802

NAME: Prisha Patel

BRANCH: CSE Core

SEMESTER: Fall Semester 2026-27

1. Introduction

Smart Study Planner is a Python-based command-line application designed to make everyday study planning easier. Instead of keeping subjects, assignments, deadlines, schedules and progress in different places, the application brings these activities into one simple system.

A student can add subjects, create tasks, plan study sessions, mark completed work and view progress. The project also demonstrates practical use of Python concepts such as functions, lists, dictionaries, loops, conditional statements, user input, input validation, modular programming, data storage and report generation.

Project Objectives

• Organize study subjects in one place.

• Create and manage study tasks and assignments.

• Plan study sessions in a simple schedule.

• Track completed and pending work.

• Calculate and display study progress.

• Generate a quick study summary.

• Demonstrate modular Python programming.

2. Problem Statement

Students often have several subjects, assignments, revision activities and deadlines at the same time. When this information is maintained in separate notebooks or applications, it becomes difficult to keep a clear picture of what needs to be done and how much work is complete.

The Smart Study Planner solves this by providing one place to maintain subjects, add tasks, arrange study sessions, update completion status and view reports.

3. Scope of the Project

The current project focuses on a lightweight command-line system. It manages subjects, tasks, schedules, progress, reports and saved study information. The project intentionally keeps the interface simple so the main focus remains on Python programming and modular design.

Target Users

- College students
- Students preparing for examinations
- Students managing multiple subjects
- Students who want a structured study routine

4. Functional Requirements

FR No.	Functional Requirement
FR1	Add and manage study subjects.
FR2	Create and view study tasks.
FR3	Associate tasks with subjects and study sessions.
FR4	Create and view a study schedule.
FR5	Mark tasks as completed and track progress.
FR6	Calculate overall study progress.
FR7	Generate study reports and summaries.
FR8	Save and retrieve study-planning data.
FR9	Allow the user to exit the application.

5. Non-Functional Requirements

• Usability – simple and understandable command-line interaction.

• Reliability – correct handling of subjects, tasks, schedules and progress.

• Maintainability – separate modules make changes easier.

• Error Handling – invalid input should produce a useful message.

• Performance – normal operations should execute quickly.

• Extensibility – future features can be added without redesigning everything.

6. System Architecture

The application follows a modular architecture. main.py controls the overall program flow and communicates with separate modules for subjects, tasks, scheduling, progress, reporting and storage.

• main.py – program controller and main menu

• subjects.py – subject management

• tasks.py – task creation and completion

• schedule.py – study-session planning

• progress.py – progress calculation

• reports.py – report generation

• storage.py – saving and loading data

Workflow

• Start Application

• Display Main Menu

• Select Operation

• Run Relevant Module

• Display Result

• Return to Menu / Exit

Module Interaction

main.py → subjects.py → schedule.py

main.py → tasks.py → progress.py → reports.py

main.py → storage.py

7. Module Responsibilities

Module	Responsibility
main.py	Controls the main menu, user interaction and overall program flow.
subjects.py	Adds and displays study subjects.
tasks.py	Creates tasks, displays them and marks tasks as completed.
schedule.py	Adds study sessions and displays the study schedule.
progress.py	Calculates total, completed, pending and percentage progress.
reports.py	Combines important study information into a report.
storage.py	Handles saving and loading planner data.

8. Storage Design

storage.py is used as the dedicated data-management layer. Keeping saving and loading in one place means the other modules do not need to know the details of file handling.

• Save subjects, tasks, schedule information and progress-related data.

• Load previously saved information when required.

• Provide a common interface for data persistence.

• Keep storage logic separate from user interaction.

9. Design Decisions

• Command-line interface keeps the project focused on core Python concepts.

• Modular structure divides responsibilities into separate files.

• Dedicated storage keeps persistence logic centralized.

• Separate progress and report modules keep calculations and presentation organized.

10. Technologies Used

• Python 3

• Functions and modules

• Lists and dictionaries

• Loops and conditional statements

• Command-line interface

• File/data storage

• Basic input validation

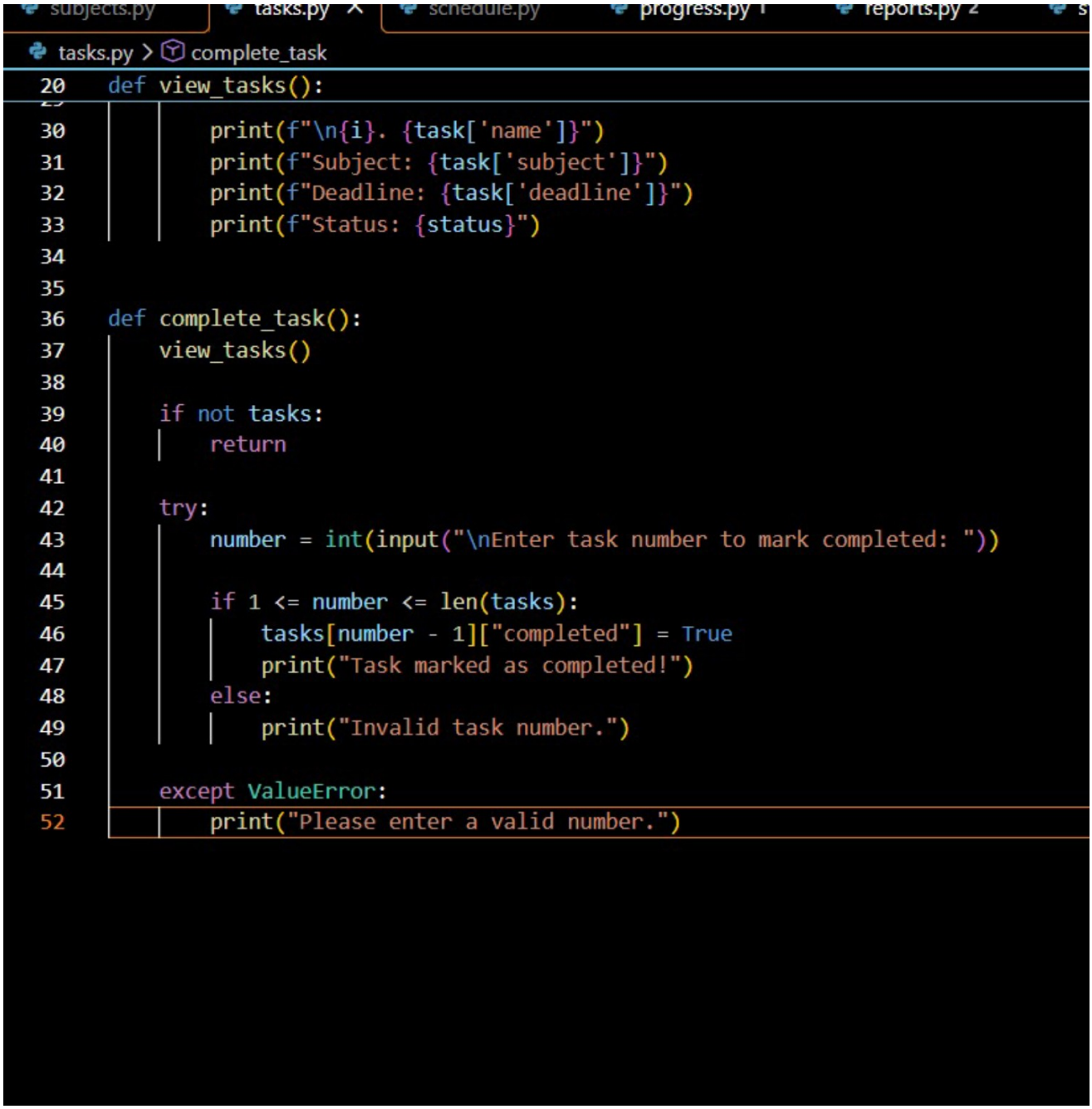
11. Code Screenshot – tasks.py



Explanation: This module collects a task name, subject and deadline from the user. These values are stored together in a dictionary. The completed field starts as False, so a newly created task is automatically treated as pending.

The view function checks whether tasks exist and then displays each task with its subject, deadline and current status. Using enumerate() gives the student a simple number for each task.

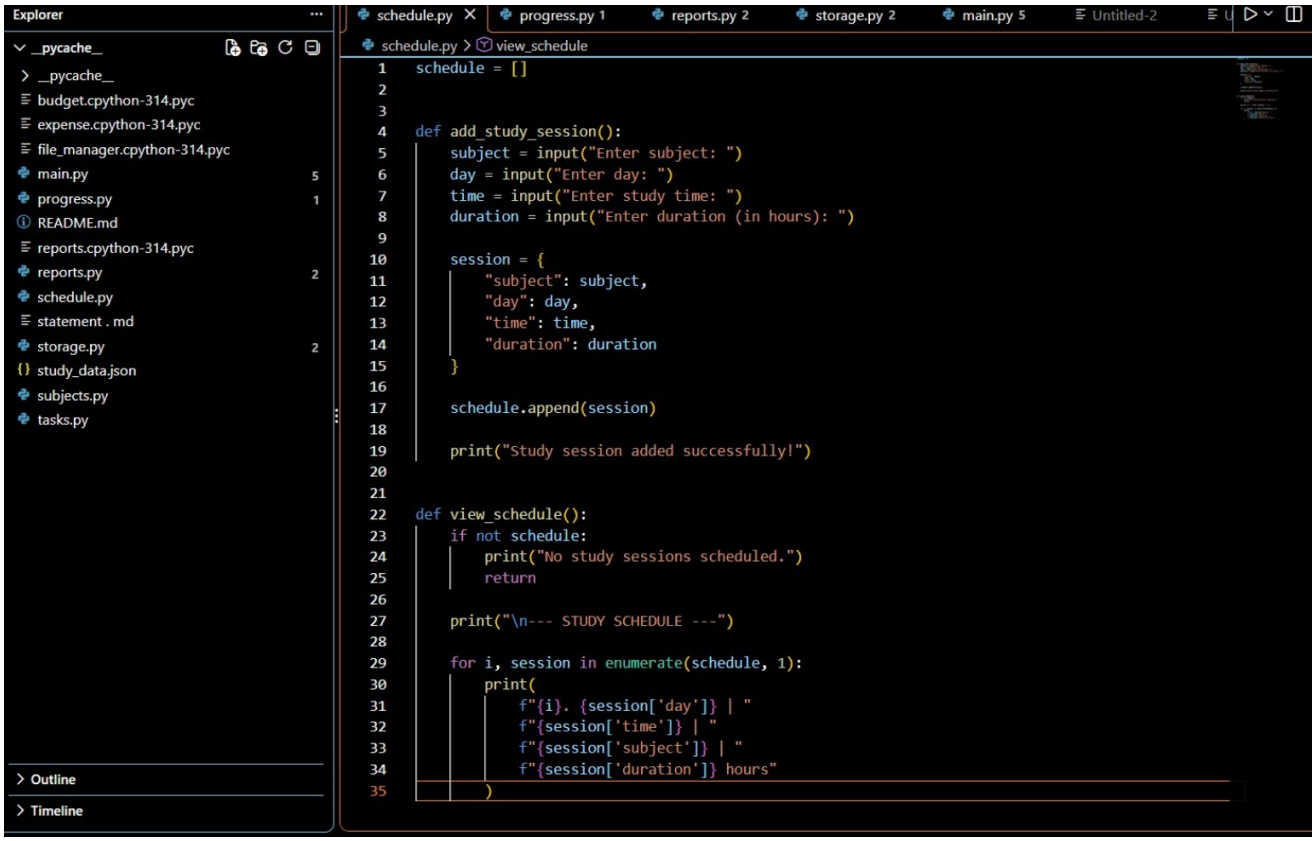
12. Code Screenshot – Completing a Task



Explanation: complete_task() first displays the available tasks. The student enters the task number they want to complete. The condition checks that the number is within the valid range before changing the selected task's completed value to True.

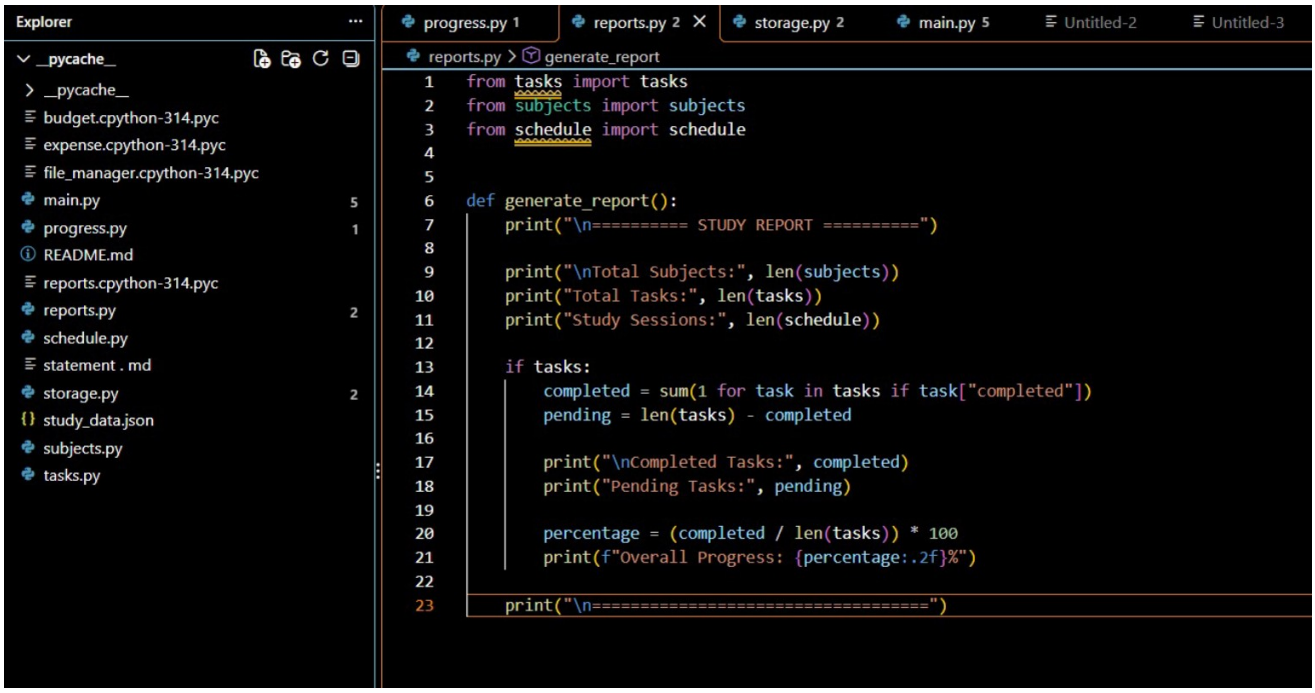
The try/except block prevents a non-numeric input from crashing the program. Instead, the user receives a simple message asking for a valid number.

13. Code Screenshot – subjects.py



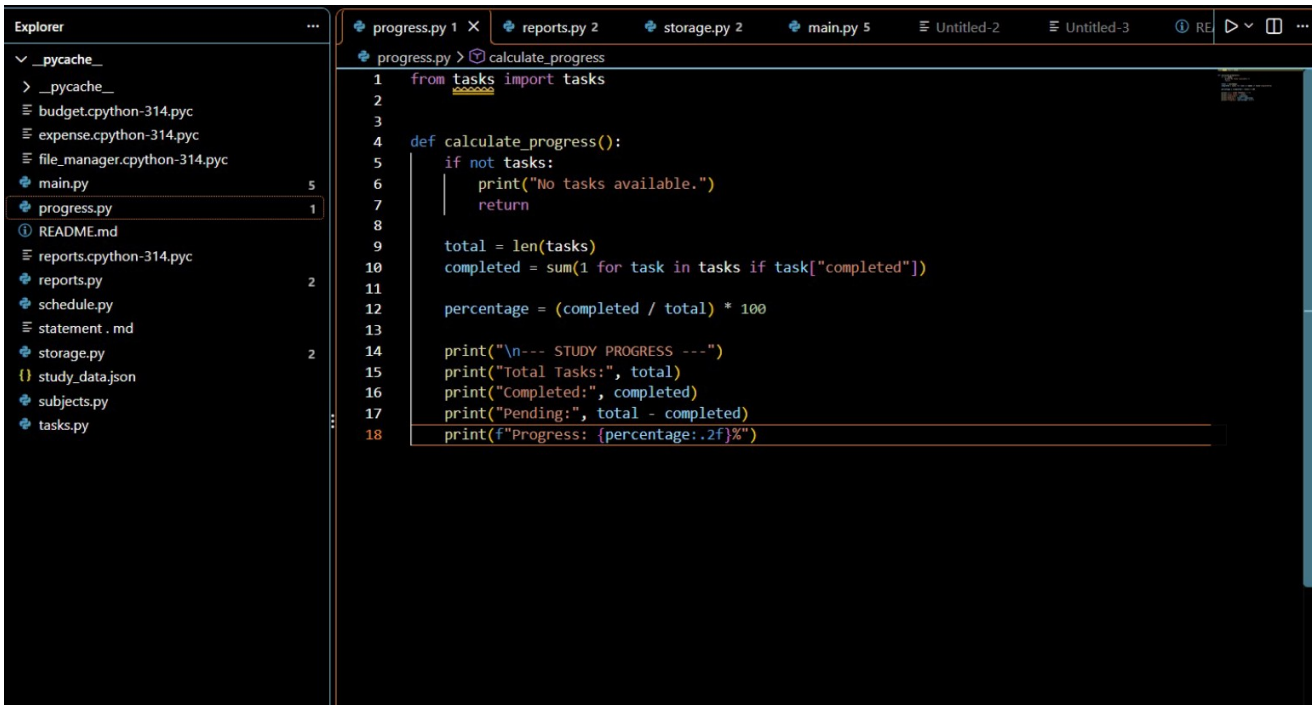
Explanation: subjects.py maintains a list of subjects. add_subject() asks for a subject name and checks whether the input is empty. A valid subject is added using append(). view_subjects() displays the subjects with numbers using enumerate().

14. Code Screenshot – schedule.py



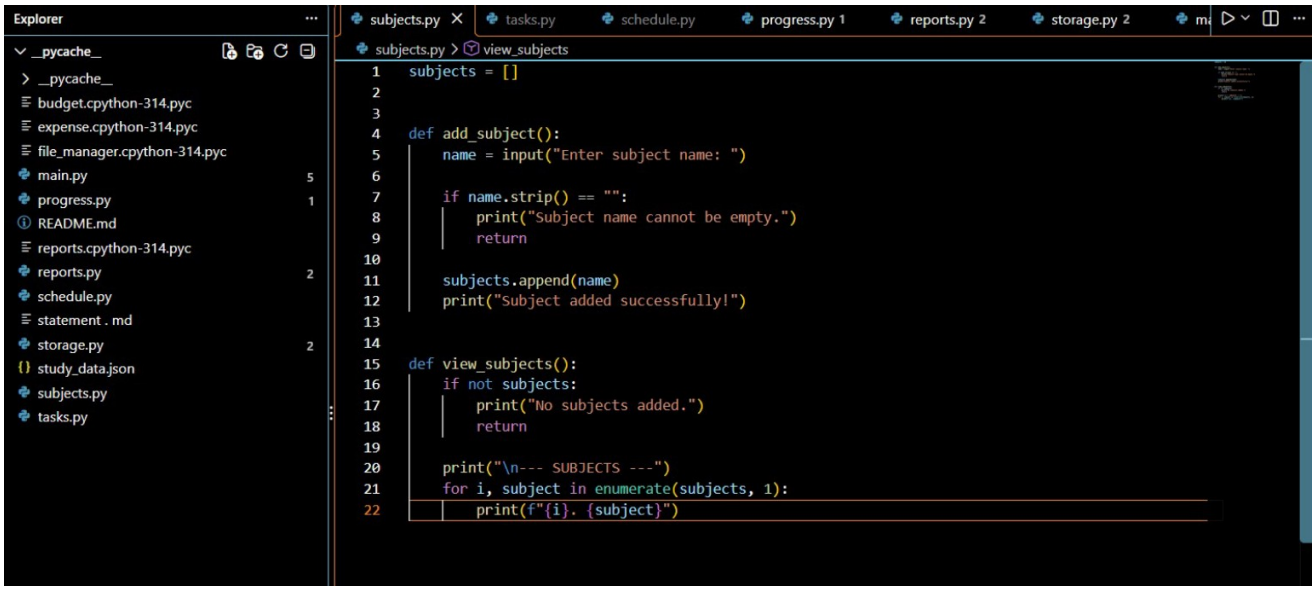
Explanation: schedule.py collects the subject, day, study time and duration. These values are grouped into a dictionary and appended to the schedule list. view_schedule() then prints the sessions in a readable format.

15. Code Screenshot – progress.py



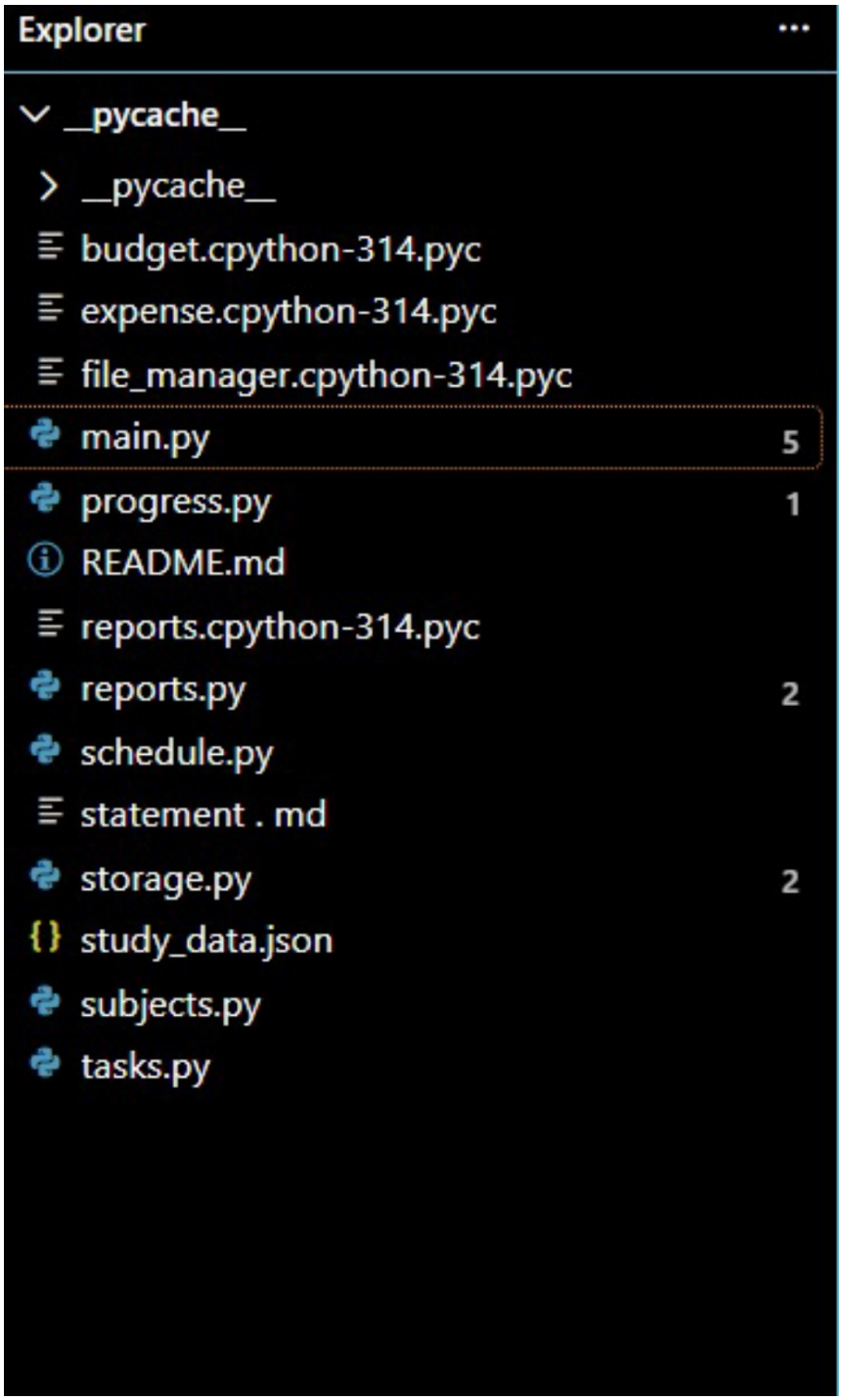
Explanation: progress.py checks whether tasks exist, counts the total number of tasks and counts completed tasks. The percentage is calculated using completed divided by total, multiplied by 100. It also displays pending tasks.

16. Code Screenshot – reports.py



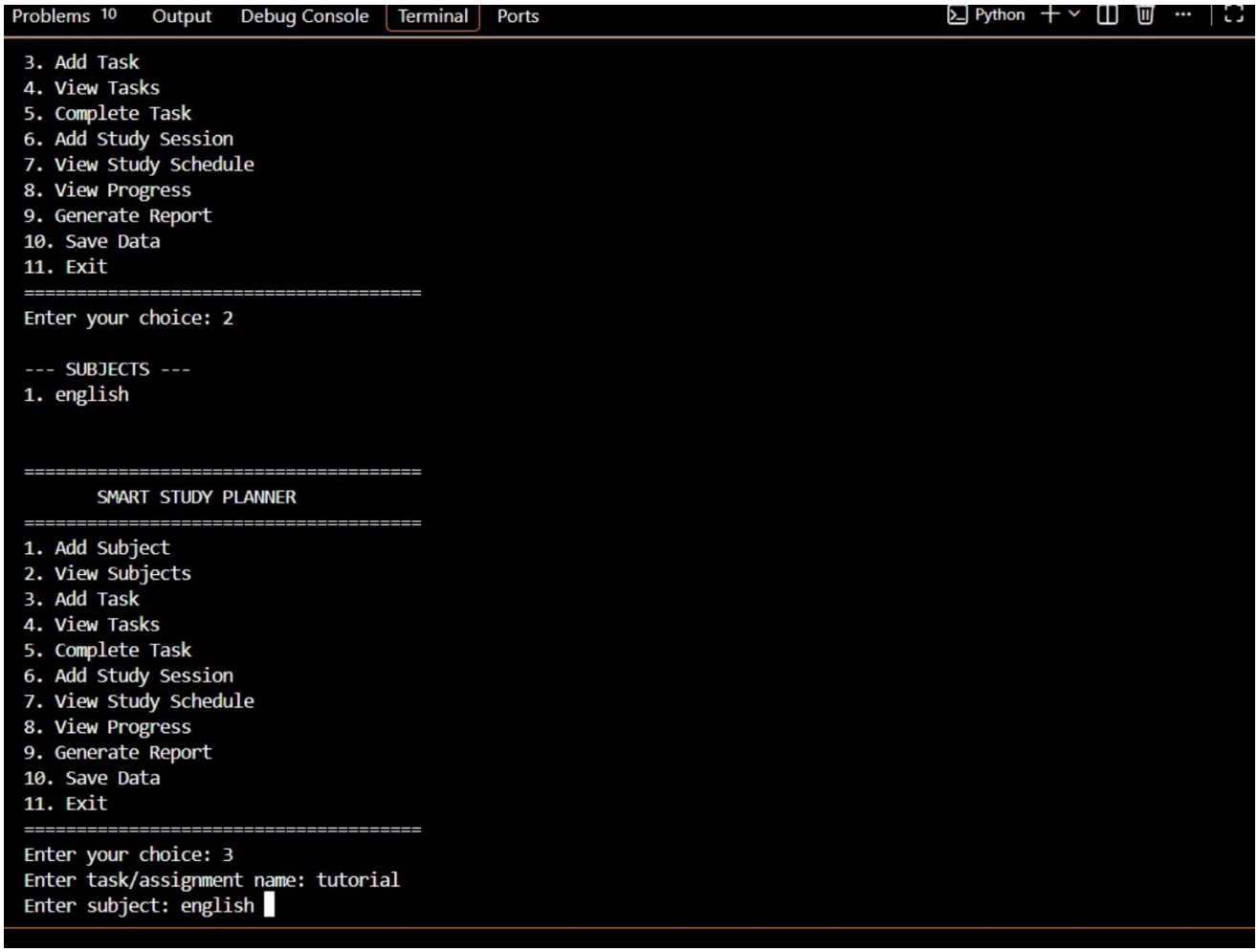
Explanation: reports.py combines information from subjects, tasks and schedule. It displays total subjects, total tasks and study sessions. If tasks exist, it also calculates completed tasks, pending tasks and overall progress.

17. Project Folder and Files



The project folder shows the main Python modules, the README and statement files, and the study_data.json file used for stored information. The seven main modules are visible: main.py, subjects.py, tasks.py, schedule.py, progress.py, reports.py and storage.py.

18. Program Output – Main Menu and Subject



The terminal output shows the Smart Study Planner menu and the View Subjects operation. In this run, the subject 'english' is displayed. The same menu then allows the user to move to Add Task and other operations.

19. Program Output – Task Added Successfully



This output demonstrates an actual task-entry flow. The user selects Add Task, enters 'tutorial' as the task name, chooses 'english' as the subject and enters the deadline 30-9-2026. The program confirms the operation with 'Task added successfully!'.

20. Testing, Learning and Conclusion

Testing was based on normal user actions such as adding a subject, creating a task, viewing tasks, completing a task, creating a study session, viewing progress, generating a report and saving data.

Suggested Tests

• Add a subject and verify it appears.

• Add a task and verify its subject and deadline.

• Complete a task and confirm its status changes.

• Create and view a study session.

• Check progress before and after completing a task.

• Generate a report and compare its totals with the stored data.

• Test invalid task numbers and empty subject input.

Learning and Key Takeaways

This project helped connect basic Python concepts with a practical student problem. The modular approach made it easier to understand what each part of the program is responsible for. It also provided practice with dictionaries, lists, functions, input validation, file storage and simple calculations.

Conclusion

The Smart Study Planner provides a simple way to organize subjects, tasks, schedules and progress through a command-line interface. Its modular structure makes the project easy to explain and provides a foundation for future additions such as reminders, a graphical interface, analytics, calendar integration and database storage.

References

• VITyarthi project guidelines.

• Python documentation and standard Python programming concepts.

• Python standard library documentation for file handling and data processing.

APPENDIX A – USER FLOW SUMMARY

The program follows a simple menu-driven flow. The student chooses an option, enters the required information, receives the result, and returns to the main menu. This makes the application easy to operate even for a first-time user.

- 1. Add Subject – creates a new subject.
- 2. View Subjects – displays the subjects currently stored.
- 3. Add Task – records a task/assignment, subject and deadline.
- 4. View Tasks – shows task details and status.
- 5. Complete Task – changes the selected task to completed.
- 6. Add Study Session – records a subject, day, time and duration.
- 7. View Study Schedule – displays planned sessions.
- 8. View Progress – calculates completed, pending and percentage progress.
- 9. Generate Report – summarizes subjects, tasks and study sessions.
- 10. Save Data – stores planner information.
- 11. Exit – closes the application.

APPENDIX B – SAMPLE RUN EXPLANATION

A sample run was captured from the actual terminal output supplied for this project.

Subject selection

The user selected View Subjects and the program displayed the subject 'english'. This confirms that subject data can be stored and displayed through the subjects module.

Task creation

The user selected Add Task and entered 'tutorial' as the task/assignment name, 'english' as the subject and 30-9-2026 as the deadline. The program then displayed 'Task added successfully!', confirming that the task was accepted.

What this demonstrates

• The main menu correctly connects the user to individual functions.

• Input is collected through input() statements.

• Task details are stored together.

• The program provides confirmation after a successful operation.

• The application returns to the main menu for the next operation.

APPENDIX C – FUTURE ENHANCEMENTS

The present version is intentionally simple, but its modular structure provides a useful base for additional features.

• Task priorities and difficulty levels.

• Automatic reminders and notifications for deadlines.

• A graphical user interface for easier interaction.

• Calendar integration for planned sessions.

• Subject-wise analytics and visual charts.

• Recurring study sessions.

• Editing and deleting existing subjects and tasks.

• Database or cloud-based storage.

• User accounts and personalized study plans.

Final Note

The report combines the project description, actual code screenshots, actual terminal outputs and plain-language explanations. The screenshots are included to make the implementation easier to understand during submission or viva.